

Mathematical Foundations — 3D Trajectory Denoising Autoencoder

Generic reference — replace symbol values per configuration

o. Variable Definitions

All quantities below are symbolic. The "Value used" column records one concrete setting; change it per experiment without altering any equation.

Symbol	Value used	Meaning	Notes
N	1000	Total trajectories in dataset	Increase for better generalisation
T	800	Timesteps per trajectory	Controls sequence resolution
σ	0.6	Noise standard deviation	Vary to test robustness
d	3	Spatial dimensions (x, y, z)	Fixed for 3D data
p	$T \times d$	Flattened input dimension	$p = 800 \times 3 = 2400$ here
H	128	Hidden layer width	Encoder & decoder share H
E	8	Embedding (bottleneck) size	Key hyperparameter to ablate
B	64	Batch size	Affects gradient variance
η	0.001	Learning rate	Adam default; try $1e-4$ to $1e-2$
Ep	400	Max training epochs	Use early stopping in practice
α	0.6	Train split fraction	$n_{\text{train}} = \alpha N, n_{\text{test}} = (1 - \alpha)N$
β_1	0.9	Adam first-moment decay	Standard default
β_2	0.999	Adam second-moment decay	Standard default
ϵ	10^{-8}	Adam numerical stability term	Prevents divide-by-zero

1. Data Representation

Each trajectory is a 3D multivariate time series — T position snapshots at equally-spaced instants. One trajectory is a matrix; the full dataset is a tensor. Before entering the autoencoder each matrix is unrolled to a flat vector:

$$\begin{aligned}\tau &\in \mathbb{R}^{T \times d} && \text{(one trajectory)} \\ X &\in \mathbb{R}^{N \times T \times d} && \text{(full dataset)} \\ x_i = \text{flatten}(\tau_i) &\in \mathbb{R}^p, \quad p = T \cdot d && \text{(flattened input)}\end{aligned}\tag{1}$$

2. Spiral Generation Function

A continuous time variable is sampled at T evenly-spaced points. The radius grows or contracts linearly; the vertical axis moves at constant speed:

$$\begin{aligned}t &= \text{linspace}(0, t_{\max}, T) \\ r(t) &= r_0 + k \cdot t \\ x(t) &= r(t) \cos(t) \\ y(t) &= r(t) \sin(t) \\ z(t) &= v_z \cdot t\end{aligned}\tag{2}$$

Parameters are drawn independently per trajectory: $r_0 \sim U(0.5, 1.5)$, $k \sim U(-0.2, 0.2)$, $v_z \sim U(-0.5, 0.5)$, $t_{\max} \sim U(2\pi, 6\pi)$. Intrinsic dimensionality = 4 (four scalars fully determine any trajectory), which justifies a bottleneck of size $E \geq 4$.

3. Noise Model

Independent Gaussian noise is added to every coordinate at every timestep:

$$\begin{aligned}\tilde{x}_i(t) &= x_i(t) + \varepsilon(t) && \text{(noisy observation)} \\ \varepsilon &\sim \mathcal{N}(0, \sigma^2) && \text{(noise distribution)}\end{aligned}\tag{3}$$

$$E[\varepsilon^2] = \sigma^2 \quad (\text{noise power})$$

$$\text{SNR (dB)} = 10 \log_{10} \left(\frac{\text{Var}(x_i)}{\sigma^2} \right)$$

4. Autoencoder Architecture

The denoising autoencoder learns $f : \mathbb{R}^p \rightarrow \mathbb{R}^p$ mapping noisy to clean trajectories via a bottleneck of dimension E .

4.1 Encoder

$$\begin{aligned} h_1 &= \text{ReLU}(W_1 \tilde{x} + b_1) & W_1 &\in \mathbb{R}^{H \times p} & (\text{layer 1}) \\ z &= \text{ReLU}(W_2 h_1 + b_2) & W_2 &\in \mathbb{R}^{E \times H} & (\text{layer 2 — bottleneck}) \end{aligned}$$

4.2 Compression ratio

$$CR = \frac{p}{E} = \frac{T \cdot d}{E} \quad (4)$$

4.3 Decoder

$$\begin{aligned} h_3 &= \text{ReLU}(W_3 z + b_3) & W_3 &\in \mathbb{R}^{H \times E} & (\text{layer 3}) \\ \hat{x} &= W_4 h_3 + b_4 & W_4 &\in \mathbb{R}^{p \times H} & (\text{output — no ReLU}) \end{aligned}$$

4.4 Full forward pass

$$\hat{x} = W_4 \cdot \text{ReLU}(W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 \tilde{x} + b_1) + b_2) + b_3) + b_4 \quad (5)$$

4.5 Total trainable parameters

$$\Phi = (pH + H) + (HE + E) + (EH + H) + (Hp + p) \quad (6)$$

Note on the output layer: no ReLU is applied after W_4 because trajectory coordinates can be negative. Applying ReLU would constrain the output to $\mathbb{R}^+$ and prevent reconstruction of negative x, y, z values.

5. Activation Function — ReLU

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

$$\text{ReLU}(v)_i = \max(0, v_i) \quad \forall i \quad (\text{element-wise})$$

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \quad (\text{derivative})$$

Why necessary: without a non-linearity, stacking Linear layers collapses —

$W_2(W_1x) = (W_2W_1)x$ — a single equivalent matrix. Spiral trajectories are non-linear in time, so ReLU is required to learn the underlying mapping.

6. Loss Function — Mean Squared Error

$$\mathcal{L}(W) = \frac{1}{N} \sum_{i=1}^N \|\hat{x}_i - x_i\|_2^2 \quad (\text{MSE loss}) \quad (8)$$

$$\mathcal{L} = \frac{1}{N \cdot p} \sum_i \sum_j (\hat{x}_{ij} - x_{ij})^2 \quad (\text{per element})$$

$$\mathcal{L}_0 \approx \text{Var}(x) + \sigma^2 \quad (\text{baseline, epoch 0})$$

MSE penalises large errors quadratically — an error of 2 units contributes $4\times$ more than an error of 1 unit — directing the optimiser to eliminate the largest noise spikes first. The baseline

$\mathcal{L}_0 \approx \text{Var}(x) + \sigma^2$ matches the loss when the model predicts the dataset mean for every input, confirming the model starts from a trivial state.

7. Training Procedure

7.1 Train / test split

$$\begin{aligned} n_{\text{train}} &= \lfloor \alpha \cdot N \rfloor && \text{(training set)} \\ n_{\text{test}} &= N - n_{\text{train}} && \text{(test set)} \end{aligned} \tag{9}$$

$$s = \left\lceil \frac{n_{\text{train}}}{B} \right\rceil \quad \text{(steps / epoch)}$$

$$U = Ep \times s = Ep \times \left\lceil \frac{n_{\text{train}}}{B} \right\rceil \quad \text{(total updates)} \tag{10}$$

The test set is locked away before training begins and used only to measure unbiased generalisation performance at the end of every epoch.

7.2 Gradient descent — one step

Each training step processes one mini-batch of B trajectories:

$$\begin{aligned} \hat{x} &= f(\tilde{x}; W) && \text{(forward)} \\ \mathcal{L} &= \text{MSE}(\hat{x}, x) && \text{(loss)} \end{aligned} \tag{11}$$

$$g_t = \frac{\partial \mathcal{L}}{\partial W} \quad \text{(gradient)}$$

$$W \leftarrow W - \Delta W \quad \text{(update)}$$

The gradient g_t is computed by backpropagation — the chain rule applied recursively from the output layer to the input layer through all four weight matrices W_4, W_3, W_2, W_1 .

7.3 Adam optimiser — detailed

Adam (Adaptive Moment Estimation) maintains two exponentially-weighted moving averages of past gradients — a first moment m (mean) and a second moment v (uncentred variance). These adapt the effective learning rate per parameter, making Adam robust to different gradient scales across layers.

Step 1 — compute gradient

$$g_t = \frac{\partial \mathcal{L}_t}{\partial W}$$

Step 2 — update first moment (gradient mean)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (12)$$

$\beta_1 = 0.9$ controls how much weight is given to the current gradient vs. the historical average. A high β_1 produces smooth, momentum-like updates.

Step 3 — update second moment (gradient variance)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (13)$$

$\beta_2 = 0.999$ tracks the squared gradient magnitude. Large v_t means high gradient variance for this parameter — Adam will reduce its effective step size.

Step 4 — bias correction

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \end{aligned} \quad (14)$$

At early timesteps, m and v are biased towards zero because they are initialised as zero vectors. Dividing by $(1 - \beta^t)$ corrects this bias — the correction factor approaches 1 as t grows and

becomes negligible.

Step 5 — parameter update

$$W_t = W_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (15)$$

The division by $\sqrt{\hat{v}_t}$ normalises the update by the parameter's historical gradient magnitude. Parameters with consistently large gradients get smaller steps; parameters with small gradients get relatively larger steps. $\epsilon = 10^{-8}$ prevents division by zero when $\hat{v}_t \approx 0$.

Effective learning rate per parameter: $\eta_{\text{eff}} \approx \eta / \sqrt{\hat{v}_t}$. If a parameter's gradient is consistently large ($\sqrt{\hat{v}_t} \gg 1$), its effective LR shrinks. If a gradient is consistently small ($\sqrt{\hat{v}_t} \approx 0$), its effective LR is close to η . This makes Adam self-tuning — it slows down in steep directions and speeds up in shallow directions, converging faster than plain SGD across the heterogeneous layers of an AE.

7.4 Early stopping

To avoid overfitting and remove the need to guess Ep , training stops when the test loss fails to improve for P consecutive epochs (patience):

$$\text{stop if } \mathcal{L}_{\text{test}}(t) \geq \mathcal{L}_{\text{test}}(t - P) \text{ for all } t \text{ in } [t - P, t] \quad (16)$$

Best practice: set Ep large (e.g. 1000) and $P = 20\text{--}50$. Save the checkpoint at the epoch with the lowest test loss, not the final epoch. The stopping epoch t^* is itself a result — report it alongside the final test MSE.

8. Why Compression Causes Denoising

The bottleneck enforces an information bottleneck between the noisy input and the clean output. Noise is i.i.d. across all p coordinates — it is incompressible into $E \ll p$ dimensions. The clean spiral signal is highly correlated across timesteps and fully determined by only 4 scalars — it is highly compressible. Minimising MSE against the clean target teaches the encoder to map trajectory structure into z and discard noise:

$$\begin{aligned}
I(z; x_{\text{clean}}) &\gg 0 && \text{(signal retained)} \\
I(z; \varepsilon) &\approx 0 && \text{(noise discarded)}
\end{aligned} \tag{17}$$

where $I(\cdot; \cdot)$ is mutual information. No explicit noise model is required — the architecture and objective together enforce the separation.

9. Evaluation Metrics

$$\mathcal{L}^* = \frac{1}{n_{\text{test}}} \sum_i \frac{\|\hat{x}_i - x_i\|^2}{p} \quad (\text{test MSE}) \tag{18}$$

$$\begin{aligned}
\text{SNR}_{\text{before}} &= 10 \log_{10} \left(\frac{\text{Var}(x)}{\sigma^2} \right) \\
\text{SNR}_{\text{after}} &= 10 \log_{10} \left(\frac{\text{Var}(x)}{\mathcal{L}^*} \right)
\end{aligned} \tag{19}$$

$$\Delta \text{SNR} = \text{SNR}_{\text{after}} - \text{SNR}_{\text{before}} \quad (\text{SNR gain})$$

$$CR = \frac{p}{E} \quad (\text{compression}) \tag{20}$$

Report $\mathcal{L}^*$, ΔSNR , CR , and the stopping epoch t^* for every configuration to enable fair comparison across experiments.